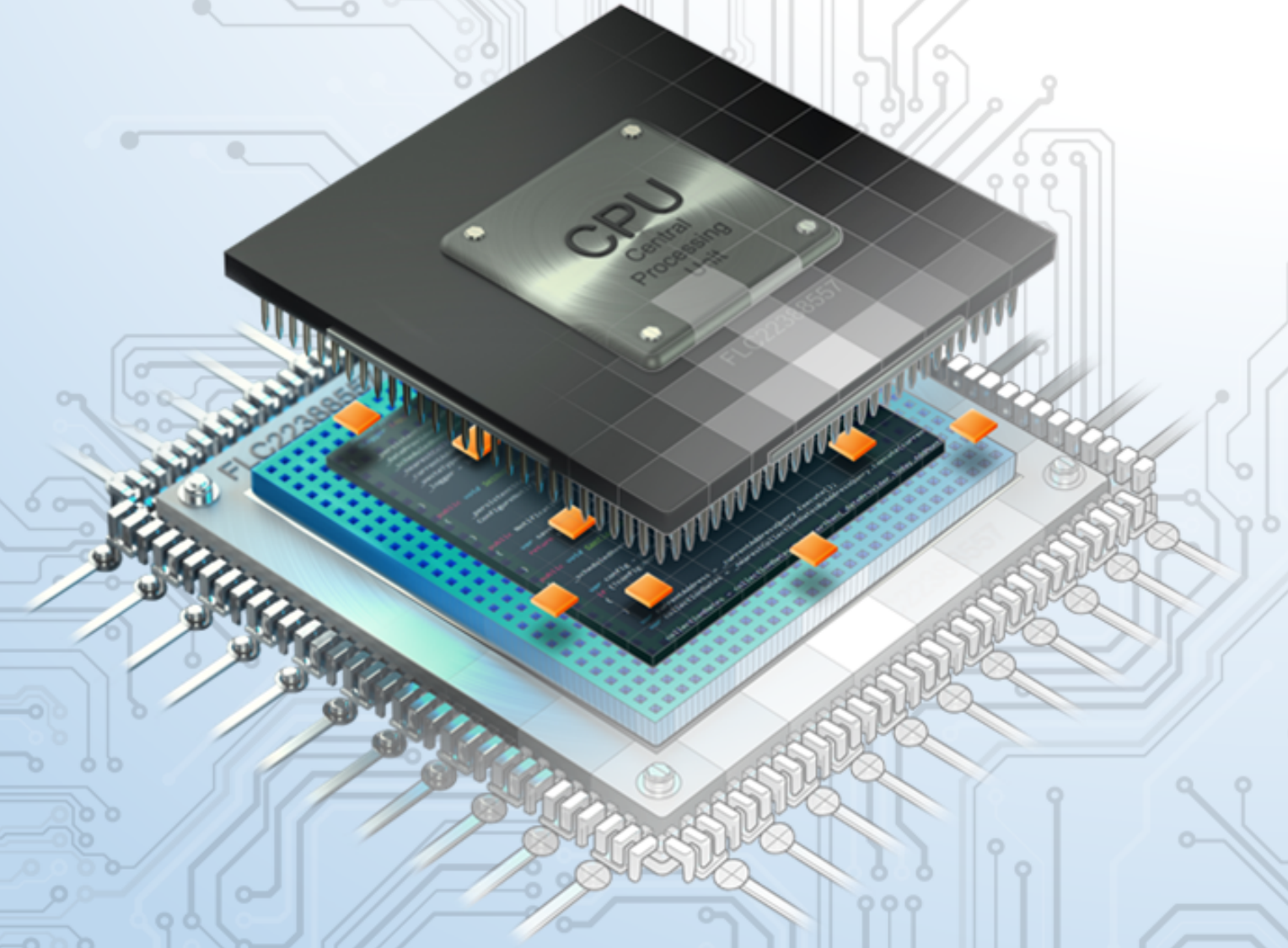
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



Dr. Le Trong Nhan

Mục lục

Chapter 1. Timer Interrupt and LED Scanning **7**

1 Introduction 8

2 Timer Interrupt Setup 10

3 Exercise and Report 13

3.1 Exercise 1 13

3.2 Exercise 2 15

3.3 Exercise 3 17

3.4 Exercise 4 19

3.5 Exercise 5 20

3.6 Exercise 6 21

3.7 Exercise 7 23

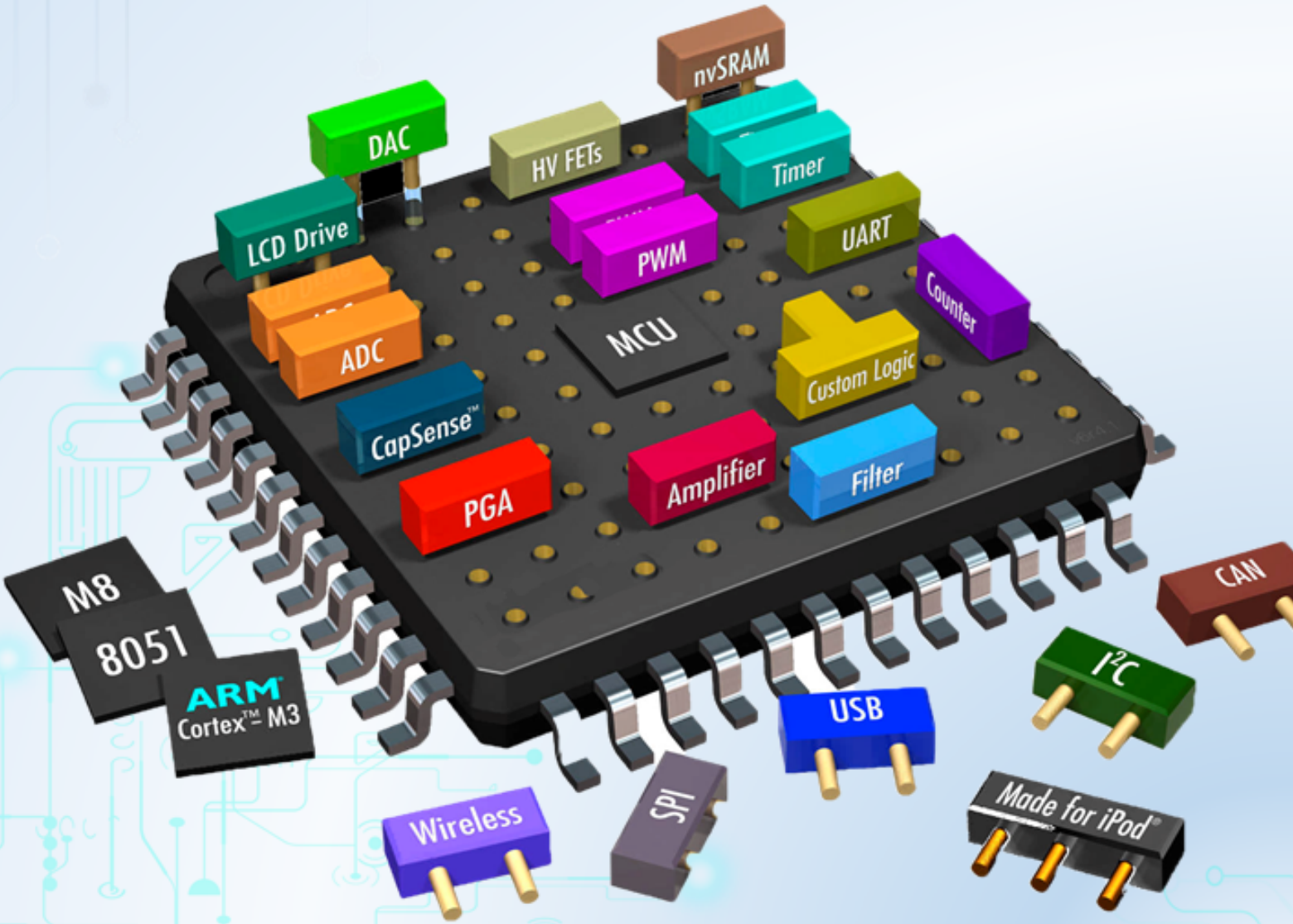
3.8 Exercise 8 24

3.9 Exercise 9 27

3.10 Exercise 10 29

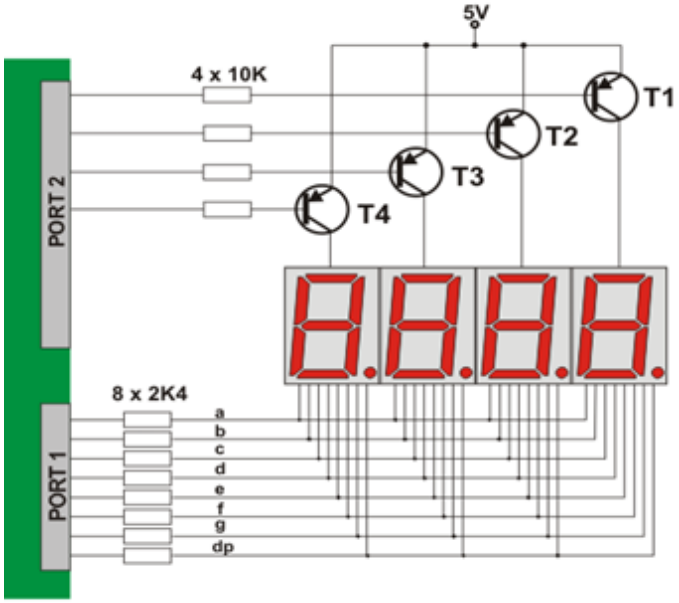
CHƯƠNG 1

Timer Interrupt and LED Scanning



1 Introduction

Timers are one of the most important features in modern micro-controllers. They allow us to measure how long something takes to execute, create non-blocking code, precisely control pin timing, and even run operating systems. In this manual, how to configure a timer using STM32CubeIDE is presented how to use them to flash an LED. Finally, students are proposed to finalize 10 exercises using timer interrupt for applications based LED Scanning.



Hình 1.1: *Four seven segment LED interface for a micro-controller*

Design an interface for with multiple LED (seven segment or matrix) displays which is to be controlled is depends on the number of input and output pins needed for controlling all the LEDs in the given matrix display, the amount of current that each pin can source and sink and the speed at which the micro-controller can send out control signals. With all these specifications, interfacing can be done for 4 seven segment LEDs with a micro-controller is proposed in the figure above.

In the above diagram each seven segment display is having 8 internal LEDs, leading to the total number of LEDs is 32. However, not all the LEDs are required to turn ON, but one of them is needed. Therefore, only 12 lines are needed to control the whole 4 seven segment LEDs. By controlling with the micro-controller, we can turn ON an LED during a same interval T_S. Therfore, the period for controlling all 4 seven segment LEDs is $4T_S$. In other words, these LEDs are scanned at frequecy $f = 1/4T_S$. Finally, it is obviously that if the frequency is greater than 30Hz (e.g. f = 50Hz), it seems that all LEDs are turn ON at the same time.

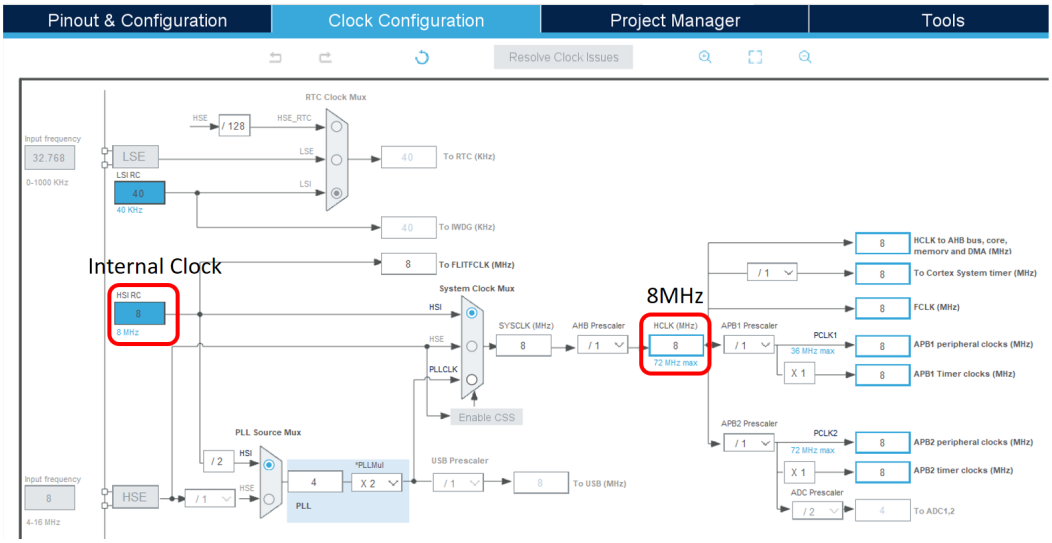
In this manual, the timer interrupt is used to design the interval T_S for LED scanning. Unfortunately, the simulation on Proteus can not execute at high frequency, the frequency f is set to a low value (e.g. 1Hz). In a real implementation, this fre-

quency should be 50Hz.

2 Timer Interrupt Setup

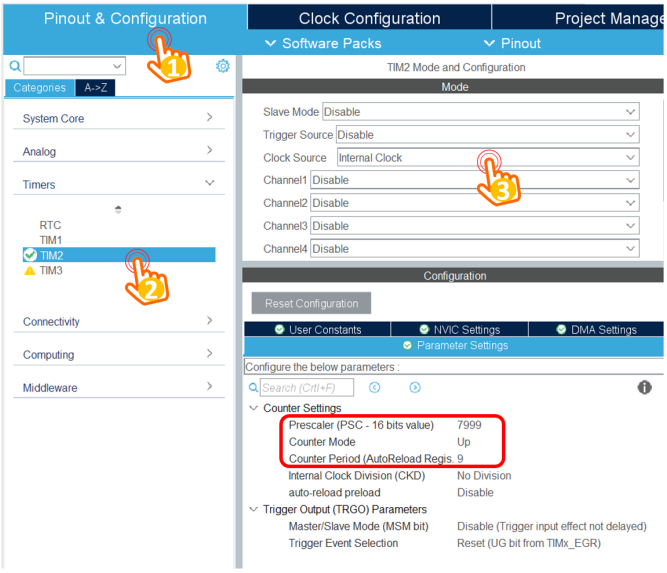
Step 1: Create a simple project, which LED connected to PA5. The manual can be found in the first lab.

Step 2: Check the clock source of the system on the tab **Clock Configuration** (from *.ioc file). In the default configuration, the internal clock source is used with 8MHz, as shown in the figure bellow.



Hình 1.2: Default clock source for the system

Step 3: Configure the timer on the **Parameter Settings**, as follows:

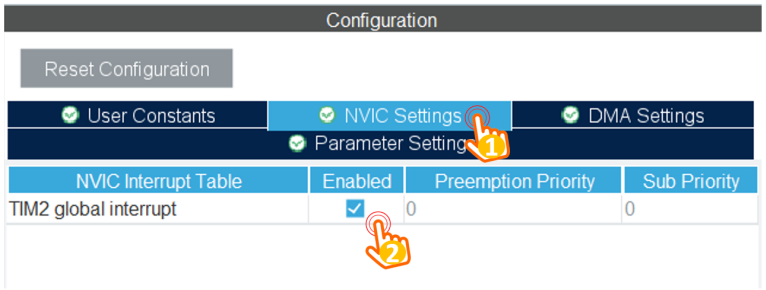


Hình 1.3: Configure for Timer 2

Select the clock source for timer 2 to the **Internal Clock**. Finally, set the prescaller and the counter to 7999 and 9, respectively. These values are explained as follows:

- The target is to set an interrupt timer to 10ms
- The clock source is 8MHz, by setting the prescaller to 7999, the input clock source to the timer is **8MHz/(7999+1) = 1000Hz**.
- The interrupt is raised when the timer counter is counted from 0 to 9, meaning that the frequency is divided by 10, which is 100Hz.
- The frequency of the timer interrupt is 100Hz, meaning that the period is **1/100Hz = 10ms**.

Step 4: Enable the timer interrupt by switching to **NIVC Settings** tab, as follows:



Hình 1.4: Enable timer interrupt

Finally, save the configuration file to generate the source code.

Step 5: On the **main()** function, call the timer init function, as follows:

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_TIM2_Init();

  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */g3

  while (1){

  }
}
```

Program 1.1: Init the timer interrupt in main

Please put the init function in a right place to avoid conflicts when code generation is executed (e.g. ioc file is updated).

Step 6: Add the interrupt service routine function, this function is invoked every 10ms, as follows:

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
   {

}
/* USER CODE END 4 */
```

Program 1.2: Add an interrupt service routine

Step 7: To run a LED Blinky demo using interrupt, a short manual is presented as follows:

```
/* USER CODE BEGIN 4 */
int counter = 100;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
   {
  counter--;
  if(counter <= 0){
    counter = 100;
    HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
  }
}
/* USER CODE END 4 */
```

Program 1.3: LED Blinky using timer interrupt

The **HAL_TIM_PeriodElapsedCallback** function is an infinite loop, which is invoked every cycle of the timer 2, in this case, is 10ms.

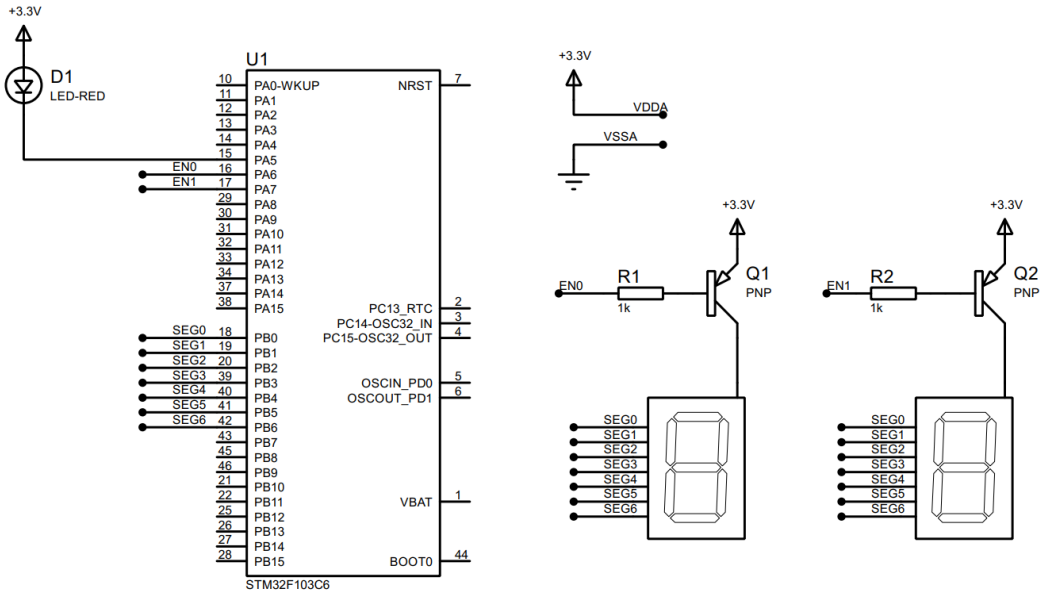
3 Exercise and Report

3.1 Exercise 1

The first exercise show how to interface for multiple seven segment LEDs to STM32F103C6 micro-controller (MCU). Seven segment displays are common anode type, meaning that the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins.

In order to save the resource of the MCU, individual cathode pins from all the seven segment LEDs are connected together, and connect to 7 pins of the MCU. These pins are popular known as the **signal pins**. Meanwhile, the anode pin of each seven segment LEDs are controlled under a power enabling circuit, for instance, an PNP transistor. At a given time, only one seven segment LED is turned on. However, if the delay is small enough, it seems that all LEDs are enabling.

Implement the circuit simulation in Proteus with two 7-SEGMENT LEDs as following:



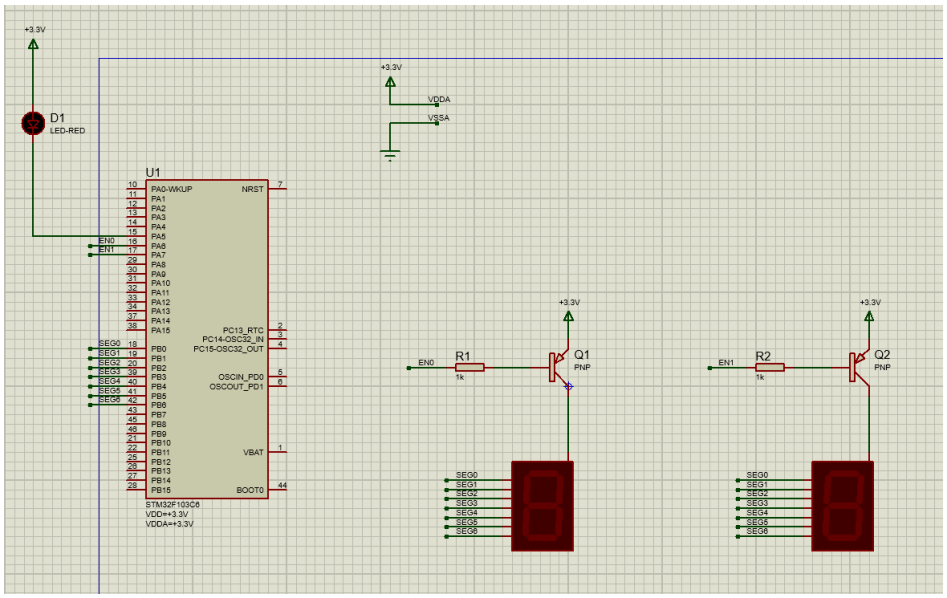
Hình 1.5: *Simulation schematic in Proteus*

Components used in the schematic are listed bellow:

- 7SEG-COM-ANODE (connected from PB0 to PB6)
- LED-RED
- PNP
- RES
- STM32F103C6

Students are proposed to use the function **display7SEG(int num)** in the Lab 1 in this exercise. Implement the source code in the interrupt callback function to display number **"1"** on the first seven segment and number **"2"** for second one. The switching time between 2 LEDs is half of second.

Report 1: Capture your schematic from Proteus and show in the report.



Hình 1.6:

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

```
/* USER CODE BEGIN 4 */
int counter = 100;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        timerCounter++;

        if (timerCounter >= 50U)
        {
            timerCounter = 0U;
            update7SEG_EX1(
                currentDigit,
                currentNumber
            );

            currentDigit++;

            if (currentDigit >= 2U)
```

```
            {
                currentDigit = 0U;
                currentNumber++;

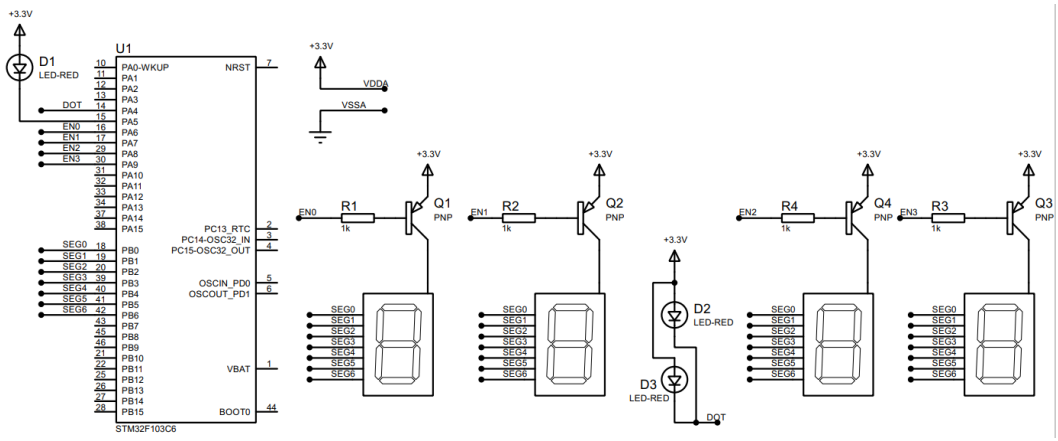
                if (currentNumber >= 10U)
                {
                    currentNumber = 0U;
                }
            }
        }
    }
}
/* USER CODE END 4 */
```

Short question: What is the frequency of the scanning process?

- Each time switching between 2 LEDs: 0.5 s -> switching frequency = 2 Hz.
- A full cycle (LED1 0.5 s + LED2 0.5 s) = 1 s -> frame/scan frequency = 1 Hz.
- Depending on the lab's definition: usually the full frame scan frequency is called 1 Hz in this task.

3.2 Exercise 2

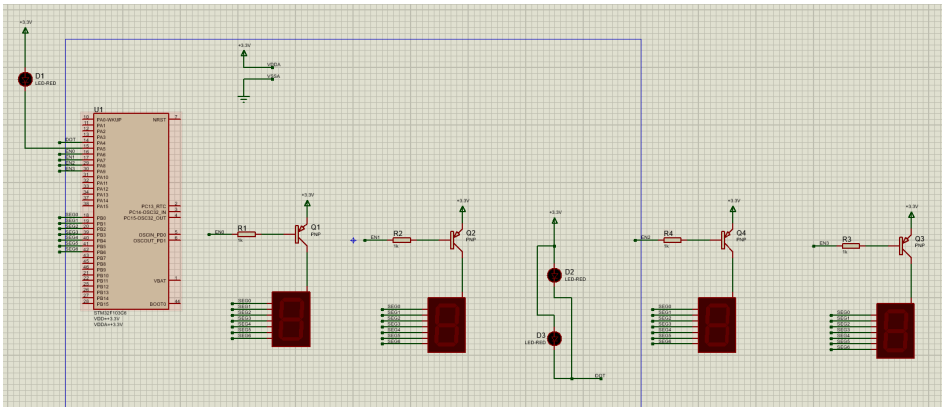
Extend to 4 seven segment LEDs and two LEDs (connected to PA4, labeled as **DOT**) in the middle as following:



Hình 1.7: *Simulation schematic in Proteus*

Blink the two LEDs every second. Meanwhile, number 3 is displayed on the third seven segment and number 0 is displayed on the last one (to present 12 hour and a half). The switching time for each seven segment LED is also a half of second (500ms). **Implement your code in the timer interrupt function.**

Report 1: Capture your schematic from Proteus and show in the report.



Hình 1.8:

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

Short question: What is the frequency of the scanning process?

- Duration each digit is lit: Ts = 500 ms = 0.5s
- One cycle comprises four digits: Tscan = 4 x 0.5 = 2 s
- Therefore, the scan cycle frequency: fscan = 1 / 2 = 0.5 Hz
- Digit switching frequency: fswitch = 1 / 0.5 = 2 Hz

Each 7-segment LED is activated for 500 ms, so the system switches digits at a frequency of 2 Hz. A full scan cycle across the four LEDs takes 2 seconds; therefore, the scan frequency for all four LEDs is 0.5 Hz.

3.3 Exercise 3

Implement a function named **update7SEG(int index)**. An array of 4 integer numbers are declared in this case. The code skeleton in this exercise is presented as following:

```
const int MAX_LED = 4;
int index_led = 0;
int led_buffer[4] = {1, 2, 3, 4};
void update7SEG(int index){
    switch (index){
        case 0:
            //Display the first 7SEG with led_buffer[0]
            break;
        case 1:
            //Display the second 7SEG with led_buffer[1]
            break;
        case 2:
            //Display the third 7SEG with led_buffer[2]
            break;
        case 3:
            //Display the forth 7SEG with led_buffer[3]
            break;
        default:
            break;
    }
}
```

Program 1.4: An example for your source code

This function should be invoked in the timer interrupt, e.g update7SEG(index_led++). The variable **index_led** is updated to stay in a valid range, which is from 0 to 3.

Report 1: Present the source code of the update7SEG function.

```
static void update7SEG(uint8_t index)
{
    if (index > MAX_LED)
    {
        return;
    }

    disableAllDigits();
    display7SEG(ledBuffer[index]);
    HAL_GPIO_WritePin(
        GPIOA,
        digitEnablePins[index],
        GPIO_PIN_RESET
    );
}
```

Report 2: Present the source code in the HAL_TIM_PeriodElapsedCallback.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        scan_counter++;
        dot_counter++;

        if (scan_counter >= SCAN_TICKS)
        {
            scan_counter = 0U;
            index_led++;

            if (index_led >= MAX_LED)
            {
                index_led = 0U;
            }

            update7SEG(index_led);
        }

        if (dot_counter >= DOT_TICKS)
        {
            dot_counter = 0U;

            HAL_GPIO_TogglePin(
                GPIOA,
                GPIO_PIN_4
            );
        }
    }
}
```

index_led runs within the range of 0 to 3.

Students are proposed to change the values in the **led_buffer** array for unit test this function, which is used afterward.

3.4 Exercise 4

Change the period of invoking update7SEG function in order to set the frequency of 4 seven segment LEDs to 1Hz. The DOT is still blinking every second.

Report 1: Present the source code in the **HAL_TIM_PeriodElapsedCallback**.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        scan_counter++;
        dot_counter++;

        if (scan_counter >= SCAN_TICKS)
        {
            scan_counter = 0U;
            index_led++;

            if (index_led >= MAX_LED)
            {
                index_led = 0U;
            }

            update7SEG(index_led);
        }

        if (dot_counter >= DOT_TICKS)
        {
            dot_counter = 0U;

            HAL_GPIO_TogglePin(
                GPIOA,
                GPIO_PIN_4
            );
        }
    }
}
```

- TIM2 interrupt = 10 ms
- Full scan cycle frequency = 1 Hz
- Cycle period = 1 s
- Time per digit = 1 s / 4 = 250 ms
- Number of interrupts = 250 / 10 = 25

3.5 Exercise 5

Implement a digital clock with **hour** and **minute** information displayed by 2 seven segment LEDs. The code skeleton in the **main** function is presented as follows:

```
int hour = 15, minute = 8, second = 50;
while(1){
    second++;
    if (second >= 60){
        second = 0;
        minute++;
    }
    if(minute >= 60){
        minute = 0;
        hour++;
    }
    if(hour >=24){
        hour = 0;
    }
    updateClockBuffer();
    HAL_Delay(1000);
}
```

Program 1.5: An example for your source code

The function **updateClockBuffer** will generate values for the array **led_buffer** according to the values of hour and minute. In the case these values are 1 digit number, digit 0 is added.

Report 1: Present the source code in the **updateClockBuffer** function.

```
static void updateClockBuffer(void)
{
    ledBuffer[0] = hour / 10U;
    ledBuffer[1] = hour % 10U;

    ledBuffer[2] = minute / 10U;
    ledBuffer[3] = minute % 10U;
}
```

3.6 Exercise 6

The main target from this exercise to reduce the complexity (or reduce code processing) in the timer interrupt. The time consumed in the interrupt can lead to the nested interrupt issue, which can crash the whole system. A simple solution can disable the timer whenever the interrupt occurs, the enable it again. However, the real-time processing is not guaranteed anymore.

In this exercise, a software timer is created and its counter is count down every timer interrupt is raised (every 10ms). By using this timer, the **Hal_Delay(1000)** in the main function is removed. In a MCU system, non-blocking delay is better than blocking delay. The details to create a software timer are presented bellow. The source code is added to your current program, **do not delete the source code you have on Exercise 5.**

Step 1: Declare variables and functions for a software timer, as following:

```
/* USER CODE BEGIN 0 */
int timer0_counter = 0;
int timer0_flag = 0;
int TIMER_CYCLE = 10;
void setTimer0(int duration){
  timer0_counter = duration /TIMER_CYCLE;
  timer0_flag = 0;
}
void timer_run(){
  if(timer0_counter > 0){
    timer0_counter--;
    if(timer0_counter == 0) timer0_flag = 1;
  }
}
/* USER CODE END 0 */
```

Program 1.6: Software timer based timer interrupt

Please change the **TIMER_CYCLE** to your timer interrupt period. In the manual code above, it is **10ms**.

Step 2: The **timer_run()** is invoked in the timer interrupt as following:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
   {

  timer_run();

  //YOUR OTHER CODE
}
```

Program 1.7: Software timer based timer interrupt

Step 3: Use the timer in the main function by invoked setTimer0 function, then

check for its flag (timer0_flag). An example to blink an LED connected to PA5 using software timer is shown as follows:

```
1 setTimer0(1000);
2 while (1){
3     if(timer0_flag == 1){
4         HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
5         setTimer0(2000);
6     }
7 }
```

Program 1.8: Software timer is used in main fuction to blink the LED

Report 1: if in line 1 of the code above is miss, what happens after that and why?
Missing `setTimer0(1000)`

Initially:

```
timer0_counter = 0
timer0_flag    = 0
```

The `timer_run()` function does not decrement the counter because `timer0_counter` is already equal to 0. Therefore, `timer0_flag` never becomes 1, and the LED does not change its state.

Report 2: if in line 1 of the code above is changed to setTimer0(1), what happens after that and why?
Using `setTimer0(1)`

Due to integer division:

$$\frac{1}{10} = 0$$

Therefore, the counter remains equal to 0, and the timer does not run.
Report 3: if in line 1 of the code above is changed to setTimer0(10), what is changed compared to 2 first questions and why?
Using `setTimer0(10)`

In this case:

$$\frac{10}{10} = 1$$

After the first timer interrupt:

```
counter: 1 -> 0
flag:    0 -> 1
```

Therefore, the task is executed after approximately 10 ms.

3.7 Exercise 7

Upgrade the source code in Exercise 5 (update values for hour, minute and second) by using the software timer and remove the HAL_Delay function at the end. Moreover, the DOT (connected to PA4) of the digital clock is also moved to main function.

Report 1: Present your source code in the while loop on main function.

```
  while (1)
  {
      if (timer0_flag == 1U)
      {
          increaseClockOneSecond();
          updateClockBuffer();

          HAL_GPIO_TogglePin(
              GPIOA,
              GPIO_PIN_4
          );

          setTimer0(1000U);
  }
}
```

The timer operates according to the following sequence:

1. The TIM2 interrupt periodically decreases `timer0_counter`.
2. After 100 timer interrupts, `timer0_counter` reaches zero.
3. The interrupt handler sets `timer0_flag = 1`.
4. The main loop detects the flag, updates the clock value, and toggles the DOT state.
5. Finally, the main loop calls `setTimer0(1000)` to start the next timing cycle.

3.8 Exercise 8

Move also the update7SEG() function from the interrupt timer to the main. Finally, the timer interrupt only used to handle software timers. All processing (or complex computations) is move to an infinite loop on the main function, optimizing the complexity of the interrupt handler function.

Report 1: Present your source code in the the main function. In the case more extra functions are used (e.g. the second software timer), present them in the report as well.

```
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration
   ---------------------------------------------------------
   */

  /* Reset of all peripherals, Initializes the Flash
   interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_TIM2_Init();
  /* USER CODE BEGIN 2 */
  disableAllDigits();

  HAL_GPIO_WritePin(
      GPIOA,
      GPIO_PIN_4,
      GPIO_PIN_SET
  );

```

```
  updateClockBuffer();

  index_led = 0U;
  update7SEG(index_led);


  setTimer0(1000U);

  setTimer1(250U);

  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
      if (timer0_flag == 1U)
      {
          increaseClockOneSecond();
          updateClockBuffer();

          HAL_GPIO_TogglePin(
              GPIOA,
              GPIO_PIN_4
          );

          setTimer0(1000U);
      }

      if (timer1_flag == 1U)
      {
          index_led++;

          if (index_led >= MAX_LED)
          {
              index_led = 0U;
          }

          update7SEG(index_led);

          setTimer1(250U);
      }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

```c
static void setTimer0(uint16_t duration)
{
    timer0_counter = duration / TIMER_CYCLE;
    timer0_flag = 0U;
}

static void setTimer1(uint16_t duration)
{
    timer1_counter = duration / TIMER_CYCLE;
    timer1_flag = 0U;
}

static void timer_run(void)
{
    if (timer0_counter > 0U)
    {
        timer0_counter--;

        if (timer0_counter == 0U)
        {
            timer0_flag = 1U;
        }
    }

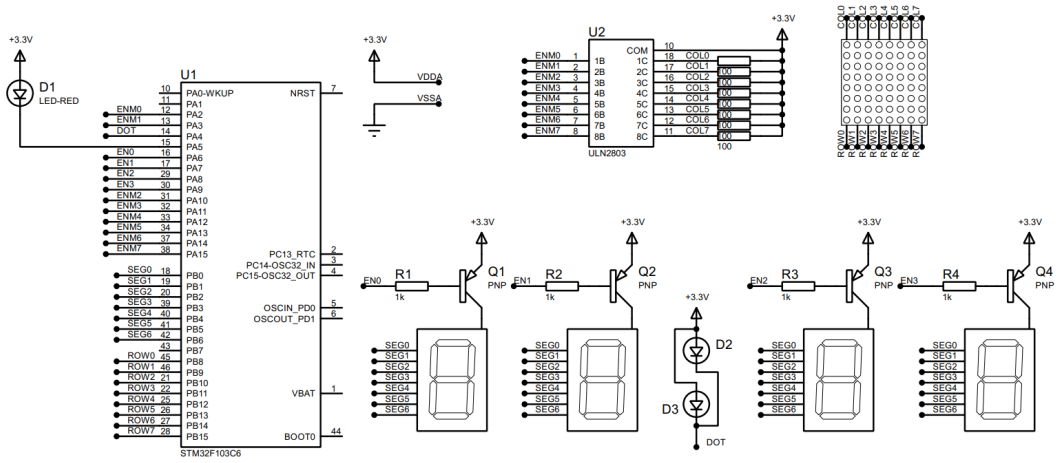
    if (timer1_counter > 0U)
    {
        timer1_counter--;

        if (timer1_counter == 0U)
        {
            timer1_flag = 1U;
        }
    }
}
```

GitHub Source Code

3.9 Exercise 9

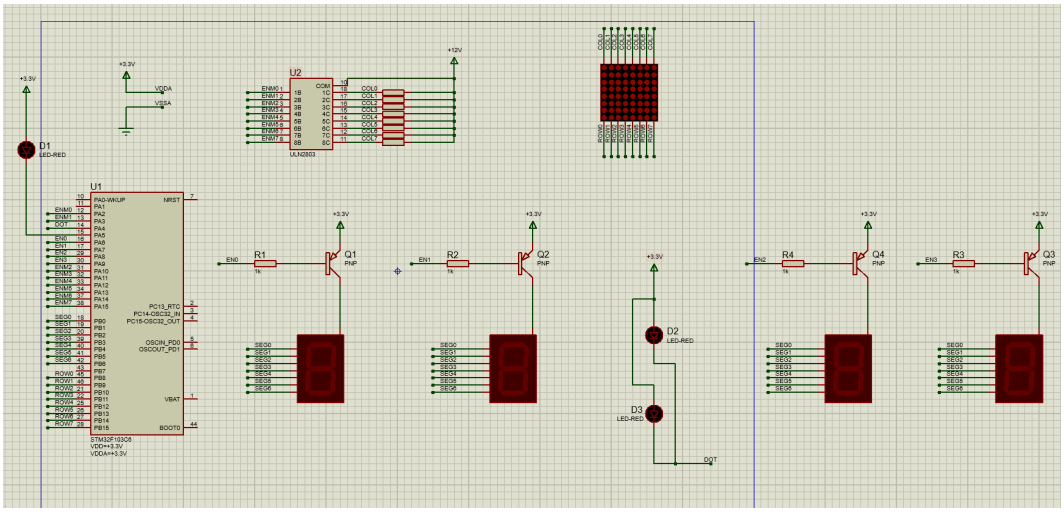
This is an extra works for this lab. A LED Matrix is added to the system. A reference design is shown in figure bellow:



Hình 1.9: LED matrix is added to the simulation

In this schematic, two new components are added, including the **MATRIX-8X8-RED** and **ULN2803**, which is an NPN transistor array to enable the power supply for a column of the LED matrix. Students can change the enable signal (from ENM0 to ENM7) if needed. Finally, the data signal (from ROW0 to ROW7) is connected to PB8 to PB15.

Report 1: Present the schematic of your system by capturing the screen in Proteus.



Report 2: Implement the function, updateLEDMatrix(int index), which is similarly to 4 seven led segments.

```
static void updateLEDMatrix(uint8_t index)
{
    uint16_t activeRowPins = 0U;

    if (index >= MAX_LED_MATRIX)
    {
        return;
    }

    /* ENM active-low: SET tat tat ca cot truoc khi thay doi ROW. */
    HAL_GPIO_WritePin(
        GPIOA,
        MATRIX_EN_ALL_PINS,
        GPIO_PIN_SET
    );

    /* ROW active-low: SET tat tat ca hang. */
    HAL_GPIO_WritePin(
        GPIOB,
        MATRIX_ROW_ALL_PINS,
        GPIO_PIN_SET
    );

    /* Moi byte la du lieu 8 hang cua mot cot; bit 1 nghia la LED sang. */
    for (uint8_t row = 0U; row < MAX_LED_MATRIX; row++)
    {
        if ((matrixLetterA[index] & (1U << row)) != 0U)
        {
            activeRowPins |= matrixRowPins[row];
        }
    }

    HAL_GPIO_WritePin(GPIOB, activeRowPins, GPIO_PIN_RESET);

    /* Chi chon mot cot bang muc RESET, dung nhu ban demo. */
    HAL_GPIO_WritePin(
        GPIOA,
        matrixEnablePins[index],
        GPIO_PIN_RESET
    );
}
```

Program 1.9: Function to display data on LED Matrix

Student are free to choose the invoking frequency of this function. However, this function is supposed to invoked in main function. Finally, please update the **matrix_buffer** to display character **"A"**.

3.10 Exercise 10

Create an animation on LED matrix, for example, the character is shifted to the left.

Report 1: Briefly describe your solution and present your source code in the report. **How the Animation Works**

The buffer consists of 16 columns:

```
static const uint8_t matrixScrollData[16] =
{
    /* 8 columns for character 'A' */
    0x00, 0xFC, 0x0A, 0x09,
    0x09, 0x0A, 0xFC, 0x00,

    /* 8 blank columns */
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00
};
```

The column to display is retrieved by:

```
dataIndex = (matrixScrollOffset + index) & 0x0F;
columnData = matrixScrollData[dataIndex];
```

Every 100 ms:

```
matrixScrollOffset++;
```

Since the display window is only 8 columns wide, increasing the offset shifts the character 'A' to the left. Using `& 0x0F` instead of the modulo operator `% 16` reduces MCU computation overhead.

The offset is updated only after scanning column 7 is completed, preventing frame-tearing artifacts between frames.

Simulation Load Optimization

Legacy Version:

- Scanned every 1 ms.
- Approximately 1,000 column updates per simulation second.
- Used 4 `HAL_GPIO_WritePin()` calls per update.
- Included an 8-iteration loop to convert bitmap into GPIO states.

New Version:

- Scanned every 2 ms.
- 500 column updates per second.
- One frame takes $8 \times 2 = 16$ ms, corresponding to 62.5 Hz.
- No `HAL_GPIO_WritePin()` calls inside the scan function.
- No 8-row decoding loop.
- Only uses 3 direct BSRR register writes:

```
GPIOA->BSRR = MATRIX_EN_ALL_PINS;                // Disable previous column
GPIOB->BSRR = rowBsrr;                           // Output 8 rows simultaneously
GPIOA->BSRR = matrixEnablePins[index] << 16U;    // Select new column
```

The number of GPIO output operations is reduced from ~4,000 HAL calls/s to 1,500 register writes/s, achieving a ~62.5% reduction while completely eliminating HAL overhead.

```
static void updateLEDMatrix(uint8_t index)
{
    uint8_t dataIndex;
    uint8_t columnData;
    uint32_t rowBsrr;

    if (index >= MAX_LED_MATRIX)
    {
        return;
    }

    dataIndex = (uint8_t)((matrixScrollOffset + index) & MATRIX_SCROLL_MASK);
    columnData = matrixScrollData[dataIndex];

    /*
     * ROW0..ROW7 nam lien tuc tai PB8..PB15 va active-low.
     * Nua duoi BSRR SET cac bit 0 cua bitmap; nua tren RESET cac bit 1.
     */
    rowBsrr = ((uint32_t)((~columnData) & 0xFFU) << 8U) |
              ((uint32_t)columnData << 24U);

    /* 3 phep ghi thanh ghi thay cho 4 HAL call va vong lap 8 lan. */
    GPIOA->BSRR = (uint32_t)MATRIX_EN_ALL_PINS;
    GPIOB->BSRR = rowBsrr;
    GPIOA->BSRR = (uint32_t)matrixEnablePins[index] << 16U;
}
```